Week 05: The Deep Learning Engine

The Math & Tools: Scalars, Vectors, Matrices, Tensors & PyTorch

WEEK 05 | TOPIC MATH & PYTORCH | DATE 14 FEB 2026

FRAMEWORK PYTORCH

*What actually happens when we say "weights * inputs"? What is a Tensor really? Today we open the hood of the Deep Learning Engine to understand the mathematics that powers everything from simple regressions to LLMs.*

Table of Contents

1 The Secret Revealed: What's Been Hiding?

> **"What were ALL of these, really?"**

In Week 2, we saw Neural Networks:

```
output = weights * inputs + bias
```

In Week 3, we saw Transformers and Attention:

```
Q = X * Wq
attention = softmax(Q * K.T) * V
```

It turns out these aren't just random variable names. They are all expressions of the same fundamental mathematical operation.

> [!IMPORTANT] **Matrix Multiplication is the secret.** Every major operation in deep learning—from a simple neuron firing to ChatGPT generating code—boils down to multiplying matrices of numbers.

The Week-by-Week Thread

Every week, we've been using the same operation without realizing it.

Week	What We Learned	The Hidden Matrix Multiplication
02	Neurons & Perceptrons	`output = input @ weights.T + bias`
03	Tokenization & Embeddings	`embedding = one_hot @ embedding_matrix`
04	Attention & Transformers	`attention = softmax(Q @ K.T) @ V`
05	**Today: We see the engine itself**	All of the above, unified.

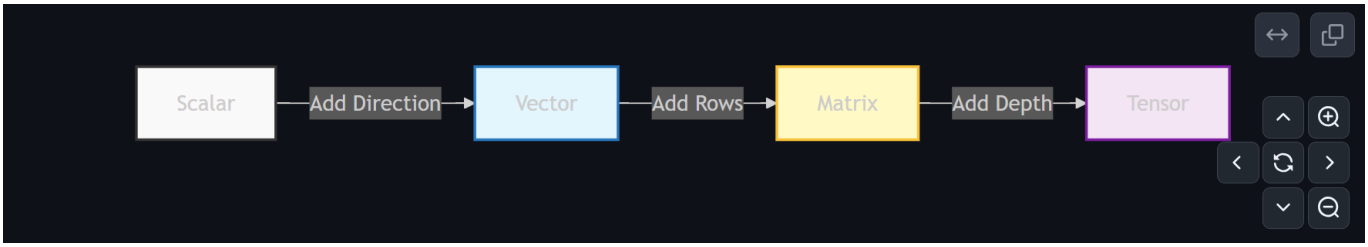
> [!TIP] **One Formula to Rule Them All:** `output = input @ weights.T + bias` — this single line IS a neural network layer.

---

2 The Hierarchy of Data: From Points to Hypercubes

We often hear the word **"Tensor"** thrown around as a scary, complex concept. It's not. It's just a generalization.

Let's build it up step-by-step.



1. Scalar: "The Simplest Thing"

Definition: A single number. A single point in space.
Dimensions: 0

- **Real-world Examples:**
  - `temperature = 72` (degrees)
  - `user_age = 25` (years)

- `coffee_price = 4.99` (dollars)

```
import torch

# 0-dimensional tensor (Scalar)
scalar = torch.tensor(20.0)
print(scalar.shape)  # torch.Size([])

# --- Useful Helpers ---
# Create a sequence: [0, 2, 4, 6, 8]
range_tensor = torch.arange(0, 10, 2)

# Create a smooth interpolation: [0.0, 0.5, 1.0]
linear_tensor = torch.linspace(0, 1, 3)

# Create zeros or ones
zeros = torch.zeros(3, 3)
ones = torch.ones(3, 3)
```

🏹 2. Vector: "Numbers with Direction"

Definition: A list of numbers. An arrow in space.
Dimensions: 1

- **Real-world Examples:**
  - **Location:** `[28.6139, 77.2090]` (Latitude, Longitude)
  - **Color:** `[255, 87, 51]` (Red, Green, Blue)
  - **Movement:** `[3, 4]` (3 steps right, 4 steps up)

> [!TIP] **Recall from Week 04:**
>
> - **Word Embeddings** (`king_vector = [0.2, -0.5, ...]`) are just **Vectors**.
> - **Inputs to a Neuron** (`features = [x1, x2, x3]`) are just **Vectors**.

🩹 The Famous Vector Math: King - Man + Woman = Queen

This is the proof that embeddings capture *meaning*. If vectors were just random numbers, this wouldn't work.

```
# In a trained embedding space:
king   = model.get_embedding("king")    # [0.5, 0.3, 0.9, ...]
man    = model.get_embedding("man")     # [0.4, 0.1, 0.6, ...]
woman  = model.get_embedding("woman")   # [0.4, 0.2, 0.7, ...]

result = king - man + woman
# result ≈ queen_vector! The "royalty" direction is preserved.
```

The vector subtraction `king - man` isolates the concept of "royalty." Adding `woman` applies that royalty to a female context. This works because each dimension in the vector encodes a **semantic feature** (gender, royalty, age, etc.).

```
# 1-dimensional tensor (Vector)
vector = torch.tensor([1.0, 2.0, 3.0, 4.0])
print(vector.shape)  # torch.Size([4])
```

▦ 3. Matrix: "The Grid"

Definition: A grid of numbers arranged in rows and columns.
Dimensions: 2

This is the workhorse of Deep Learning.

- **Key Insight:** A Matrix is a **2D Tensor**.
- **Properties:** Indexed by `(row, col)`.

💡 What Does Each Cell Mean?

In a weight matrix `W` with shape `[output_features, input_features]`:

- `W[i][j]` = **"How much does input feature `j` influence output feature `i`?"**
- A large positive value means strong positive influence.
- A value near zero means "I don't care about this input."
- A negative value means "this input pushes me the other way."

Training a neural network is literally learning the right numbers for every cell in these matrices.

> [!NOTE] **Matrices You've Already Seen:**
>
> - **Weights (W):** The connections between layers in a Neural Network.
> - **Query, Key, Value (Q, K, V):** The core components of Self-Attention.

```
# 2-dimensional tensor (Matrix)
matrix = torch.tensor([[1, 2, 3],
                       [4, 5, 6]])
print(matrix.shape)  # torch.Size([2, 3])
```

📦 4. Tensor: "The Generalization"

Definition: An n-dimensional array. A cube, a hypercube, or beyond.
Dimensions: n (0, 1, 2, 3, ...)

A "Tensor" is just the generic name for *any* of these structures.

Rank	Name	Analogy	Structure
0D	Scalar	A Point	`.`
1D	Vector	A Line	`[...]`
2D	Matrix	A Spreadsheet	`[[...], [...]]`
3D+	Tensor	A Book of Spreadsheets	`[[[...]]]`

🔘 The Dimension Cheatsheet

Memorize this table. It maps the math to the code to the real world.

Rank	Name	Code Shape Example	Physics Analogy	Data Analogy
0	Scalar	`torch.Size([])`	Mass, Temperature	Loss Value, Accuracy
1	Vector	`torch.Size([3])`	Velocity, Force	A Row in Excel, A Word Embedding
2	Matrix	`torch.Size([3, 4])`	Stress Tensor	A Black & White Image, A Spreadsheet
3	Tensor	`torch.Size([3, 224, 224])`	Magnetic Field	An RGB Image (C, H, W)
4	Tensor	`torch.Size([32, 3, 224, 224])`	Spacetime Curvature	A Batch of RGB Images (B, C, H, W)

> [!IMPORTANT] **It's not a scary new concept.** It's just: "*What if we keep adding dimensions?*"

🖼️ Unpacking an RGB Image Tensor

When you see `[3, 224, 224]`, here's exactly what each dimension means:

```
image = torch.randn(3, 224, 224)  # A single RGB image

# Dimension 0: Color Channels (3)
red_channel   = image[0]  # Shape: [224, 224] — intensity of red at each pixel
green_channel = image[1]  # Shape: [224, 224] — intensity of green
blue_channel  = image[2]  # Shape: [224, 224] — intensity of blue

# Each channel is a grayscale "heatmap" of that color.
# Stack 3 of them = full color image.
```

When you see `[32, 3, 224, 224]`, it's 32 of these images stacked together (a "batch").

```
# 3-dimensional tensor
tensor_3d = torch.zeros(2, 3, 4)
```

```
print(tensor_3d.shape)  # torch.Size([2, 3, 4])
```

---

3 Why Tensors? Structure is Everything

We need tensors because **data has structure**, and we need a mathematical object that can hold that structure.

📐 Mapping Real World Data to Tensors

Data Type	Deep Learning Representation	Tensor Shape
A Single Sentence	A Matrix of tokens	`[12 tokens, 768 dims]` (2D)
A Batch of Sentences	A Stack of matrices	`[8 sentences, 12 tokens, 768 dims]` (3D)
Multi-Head Attention	Split by heads	`[8 sentences, 12 heads, 3 tokens, 64 dims]` (4D)
Audio Waveform	Channels × Samples	`[2, 44100]` (stereo, 1 sec)
Video Clip	Batch × Frames × C × H × W	`[1, 30, 3, 224, 224]` (5D)
Tabular Data	Rows × Features	`[1000, 15]` (2D)

> [!TIP] **Shape is Everything.** In Deep Learning, your #1 bug will be **shape mismatch**. 90% of your time will be spent ensuring your tensor shapes align correctly for matrix multiplication (e.g., `[32, 768]` x `[768, 10]`).

🔌 The #1 Debugging Tool

When in doubt, **print the shape**. This is your single most powerful debugging technique.

```
x = torch.randn(32, 784)        # Input batch
print(f"Input:  {x.shape}")      # [32, 784]

W = torch.randn(784, 256)       # Weight matrix
print(f"Weight: {W.shape}")      # [784, 256]

out = x @ W
print(f"Output: {out.shape}")    # [32, 256] ☑ Inner dims matched!
```

---

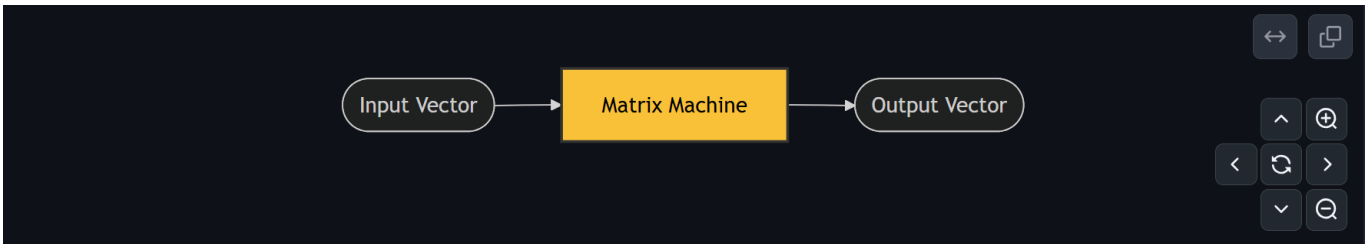
4 Matrix Transformations: The "Magic"

We established that matrices are grids of numbers. But in Deep Learning, we don't just "store" numbers in matrices. We use matrices to **change** vectors.

A Matrix is a Machine

Think of a matrix not as a dataset, but as a **function** or a **machine**.

- **Input:** A vector (a point in space).
- **The Machine:** The Matrix.
- **Output:** A new vector (a new point in space).



What Can This Machine Do?

It can transform space in specific ways:

1. **Rotate:** Spin the vector.
2. **Scale:** Grow or shrink the vector.
3. **Shear:** Slant the space.
4. **Reflect:** Flip the vector.
5. **Project:** Smash dimensions (3D -> 2D).

Visual Transformation Catalog

Here is what specific matrices do to space.

Matrix Type	What it looks like	Effect on Grid
Identity	`[[1, 0], [0, 1]]`	**Do Nothing.** The grid stays perfect.
Scale	`[[2, 0], [0, 2]]`	**Zoom In.** The grid expands by 2x.
Shear	`[[1, 1], [0, 1]]`	**Slant.** The vertical lines tilt right.
Rotation	`[[0, -1], [1, 0]]`	**Spin.** References rotate 90 degrees.
Projection	`[[1, 0], [0, 0]]`	**Flatten.** Everything collapses onto the X-axis.

> [!TIP] **Key Insight:** In a Neural Network, the "weights" are just a matrix. Training the network literally means finding the **right matrix** that rotates/stretches your input data until it's easy to classify.

Example: Rotation

Imagine a vector `[1, 0]` pointing Right. We apply a rotation matrix.

```
# Helper to visualize transformations (from class notebook)
def plot_transformation(matrix, title="Transformation"):
    # (Visualizer code omitted)
    pass
```

```
# Input: Pointing Right
vector = [1, 0]

# Matrix: Rotate 90 degrees counter-clockwise
matrix = [[0, -1],
          [1,  0]]

# Output: Pointing Up
output = [0, 1]
```

The matrix **rotated** the vector.

🤯 The Neural Network Connection

Every layer in a neural network is doing *exactly* this. Layer 1 rotates and scales your raw data. Layer 2 rotates that result again. Layer 3 rotates it again. By the end, your originally messy data points are neatly separated.

Imagine two groups of dots (cats vs dogs) tangled together. Each matrix multiplication is a **spatial transformation** that gradually un-tangles them until a simple line can separate them. That's classification.

5️⃣ Matrix Multiplication: The Core Engine

If a Matrix is a machine, **Matrix Multiplication** is the act of running the machine.

> [!IMPORTANT] **Why is Matrix Multiplication EVERYTHING?**
>
> | Concept | Math Operation | Meaning |
> | --- | --- | --- |
> | **Forward Pass** | Y = X @ W | Transforming input features into hidden states. |
> | **Attention** | A = Q @ K.T | Measuring similarity between tokens. |
> | **Embeddings** | E = OneHot @ W_emb | Looking up the meaning of a word. |
> | **Backprop** | dL/dW ... | Computing gradients (also involves matmul). |

Every single computation in a modern AI model is essentially a series of matrix multiplications.

🤯 The Dot Product: The Atomic Operation

At the lowest level, matrix multiplication is just a bunch of **dot products**.

📝 Matmul By Hand (2×2 Example)

Let's do one manually so it never feels like magic again.

```
# A = [[1, 2],    B = [[5, 6],
#      [3, 4]]         [7, 8]]
```

```
# Result[0][0] = Row 0 of A . Col 0 of B = (1*5) + (2*7) = 19
# Result[0][1] = Row 0 of A . Col 1 of B = (1*6) + (2*8) = 22
# Result[1][0] = Row 1 of A . Col 0 of B = (3*5) + (4*7) = 43
# Result[1][1] = Row 1 of A . Col 1 of B = (3*6) + (4*8) = 50

A = torch.tensor([[1, 2], [3, 4]])
B = torch.tensor([[5, 6], [7, 8]])
print(A @ B)  # tensor([[19, 22], [43, 50]]) ✔
```

Pattern: Each cell in the output is the **dot product** of one row from A and one column from B.

> [!TIP] **Dot Product = Similarity**
>
> - **High Value:** Vectors point in the **same** direction (Similar).
> - **Zero:** Vectors are **perpendicular** (Unrelated).
> - **Negative:** Vectors point in **opposite** directions (Opposite).

🔗 Connecting to Attention

When you hear "Self-Attention", it's just a Dot Product happening on a massive scale.

- **Query (Q):** "What am I looking for?" (e.g., "sat")
- **Key (K):** "What do I contain?" (e.g., "Cat")
- **Dot Product (Q . K):** "How relevant are we?"

```
# 'sat' looking for 'cat' -> HIGH Score (0.8)
score = Q_sat @ K_cat.T

# 'sat' looking for 'the' -> LOW Score (0.1)
score = Q_sat @ K_the.T
```

The model learns the matrices (W_q, W_k) that **rotate** these vectors so that related words point in the same direction!

---

6 Why GPUs Exist: Parallelism

Now that we know Deep Learning is just millions of Matrix Multiplications, we assume we need a super-fast CPU, right? **Wrong.**

🏎️ CPU vs. GPU: The Analogy

Feature	CPU (Central Processing Unit)	GPU (Graphics Processing Unit)
Architecture	Few, powerful cores (8-16)	Thousands of weak cores (5000+)
Best For	Complex, sequential logic (Branching, OS)	Simple, parallel math (Add/Multiply)
Analogy	A team of 10 PhD Mathematicians	A army of 10,000 Kindergarteners

Scenario: You need to add `1 + 1` a billion times.

- **CPU:** The PhDs do it one by one. It takes forever.
- **GPU:** The Kindergarteners each do ONE and it's done in 1 second.

💻 Hands-on: CPU vs GPU Speed Test

Run this in Colab to see the difference yourself.

```
import time
import torch

size = 2000
A = torch.randn(size, size).cuda()
B = torch.randn(size, size).cuda()

# GPU warmup (first run initializes CUDA)
A @ B

# Benchmark
start = time.time()
C = A @ B
torch.cuda.synchronize() # Wait for GPU to finish
print(f"GPU time: {time.time() - start:.5f}s")
# Result: ~0.005s (vs ~9.0s on CPU) -> 1800x Faster! 🚀
```

> [!NOTE] **Why NVIDIA is Trillion Dollar Company:** Deep Learning is NOT "complex logic". It is **simple math** (Matrix Multiplication) done **billions of times**. GPUs are architectures literally built for this exact purpose (originally for graphics pixels, now for AI tensors).

💾 CUDA Memory: The Hidden Constraint

The GPU has its own dedicated RAM called **VRAM** (Video RAM). This is separate from your system RAM.

Concept	What It Means
VRAM	The GPU's own memory (e.g., 8GB, 16GB, 24GB).
OOM Error	`RuntimeError: CUDA out of memory` — your tensors are too big for the GPU.
The Fix	Reduce batch size, use `float16`, or use gradient checkpointing.

```
# Check how much GPU memory you're using
print(f"Allocated: {torch.cuda.memory_allocated() / 1e9:.2f} GB")
print(f"Reserved:  {torch.cuda.memory_reserved() / 1e9:.2f} GB")
```

> [!WARNING] **The Device Mismatch Trap:** Both your **model** AND your **data** must be on the same device. If the model is on GPU and the data is on CPU, PyTorch will crash.

```
model = model.to("cuda")  # Model on GPU
data = data.to("cuda")    # Data MUST also be on GPU!
output = model(data)       # Now it works ☑
```

---

7 PyTorch Under the Hood

PyTorch is the standard tool for modern AI research (used by OpenAI, DeepMind, Tesla).

What is a Tensor, *Really*?

When you create `x = torch.tensor([1, 2])`, PyTorch stores two things:

1. **The Data:** The actual numbers (1, 2).
2. **The Metadata:** Shape, Device, Data Type.

The metadata tells PyTorch **how** to interpret the data.

Deep Dive: Strides & Contiguity

(Why is `.view()` instant but `.reshape()` sometimes slow?)

A Tensor is just a flat array of numbers in memory (1D). The "Shape" is an illusion created by **Strides**.

- **Stride:** How many steps in memory to jump to get to the next element in a dimension.

Example: Matrix `[2, 3]` Mem: `[1, 2, 3, 4, 5, 6]`

- To go right (Col + 1): Jump 1 step.
- To go down (Row + 1): Jump 3 steps.

When you call `.permute()`, PyTorch just creating a **new view** with different strides. It doesn't move the data! This is why it's instant. But it makes the tensor **non-contiguous** (memory jumps are messy).

```
# Proof that .view() is zero-copy (Same Memory Address)
x = torch.randn(3, 4)
y = x.view(4, 3)

print(f"Original address: {x.data_ptr()}")
print(f"Reshaped address: {y.data_ptr()}")
# Output is IDENTICAL. No data was copied!
```

Autograd: The Tape Recorder

PyTorch has a built-in engine called **Autograd**.

- `requires_grad=True`: Tells PyTorch "Start recording everything that happens to this tensor."
- `torch.no_grad()`: "Stop recording." (Used during inference/testing to save memory).

- `optimizer.zero_grad()`: **CRITICAL.** PyTorch *accumulates* gradients by default. You must wipe the slate clean before every training step, or new gradients will be added to the old ones.

⚠ The Gradient Accumulation Gotcha

```
x = torch.tensor(3.0, requires_grad=True)

# Run 1
y = x ** 2
y.backward()
print(x.grad)  # 6.0 ✔

# Run 2 (WITHOUT zeroing)
y = x ** 2
y.backward()
print(x.grad)  # 12.0 🚨 It ADDED to the old gradient!

# The fix:
x.grad.zero_()  # Always zero before backward()
```

📊 Data Types (`dtype`): Precision vs. Speed

Not all numbers are equal. PyTorch supports multiple precisions.

dtype	Size	Use Case	Trade-off
`torch.float32`	4 bytes	**Default for training.** Full precision.	Slow but accurate.
`torch.float16`	2 bytes	**Mixed-precision training.** Saves VRAM.	Fast but can overflow.
`torch.bfloat16`	2 bytes	**Modern GPUs (A100+).** Best of both worlds.	Like float16 but more stable.
`torch.int64`	8 bytes	**Labels & indices.** Not for weights.	Integer only.

```
# Convert precision to save memory
x = torch.randn(1000, 1000)          # float32: 4MB
x_half = x.half()                     # float16: 2MB (50% savings!)
print(x.dtype, x_half.dtype)          # torch.float32, torch.float16
```

---

8 Tensor Operations: The Syntax

✕ The `@` Operator (MatMul)

In Python/PyTorch, we use `@` for matrix multiplication.

```
A = torch.randn(3, 4) # Shape: [3, 4]
B = torch.randn(4, 5) # Shape: [4, 5]

# The Magic Symbol
C = A @ B
# Result Shape: [3, 5] (Inner dims '4' cancel out)
```

> [!WARNING] **The Golden Rule:** Inner dimensions MUST match. `(3, 4) @ (4, 5)` ☑ `(3, 4) @ (3, 5)` ❌ Error!

🧩 Visual Dimensions Guide

When multiplying matrices, imagine the shapes interlocking like Lego bricks.

```
   Matrix A      Matrix B      Result
[ M  x  K ]  @  [ K  x  N ]  =  [ M  x  N ]
        ^         ^
        |_________|
       MUST MATCH!
```

- **Inner Dimensions** (K, K) must be equal. They disappear.
- **Outer Dimensions** (M, N) become the shape of the result. Changing the "shape" of data without changing the data itself.

1. `.view()`: Reinterpreting Dimensions

Think of this as "flattening" or "grouping".

```
# Batch of 8, Sequence of 3, Embedding of 768
x = torch.randn(8, 3, 768)

# Flatten the batch and sequence
y = x.view(24, 768)
# (8 * 3 = 24)
```

2. `.permute()`: Swapping Axes

Crucial for Multi-Head Attention.

```
# [Batch, Tokens, Heads, Dims]
x = torch.randn(8, 12, 3, 64)

# Swap 'Tokens' (1) and 'Heads' (2)
```

```
y = x.permute(0, 2, 1, 3)
# New Shape: [8, 3, 12, 64]
```

🚀 The Magic Line: Moving to GPU

```
# 1. Check if GPU is available
device = "cuda" if torch.cuda.is_available() else "cpu"

# 2. Move tensor to device
x = x.to(device)

# Operations now happen on 5000+ cores! 🚗 -> 🚀
```

📦 Essential Shape Operations

These are the tools you'll use daily to fix shape mismatches.

3. `.squeeze()` / `.unsqueeze()`: Adding/Removing Dimensions

The most common "shape fix" operation.

```
# unsqueeze: Add a dimension of size 1
x = torch.randn(3, 4)         # [3, 4]
y = x.unsqueeze(0)             # [1, 3, 4] — Added a "batch" dimension
z = x.unsqueeze(-1)            # [3, 4, 1] — Added a trailing dimension

# squeeze: Remove all dimensions of size 1
a = torch.randn(1, 3, 1, 4)   # [1, 3, 1, 4]
b = a.squeeze()                # [3, 4] — Stripped away the 1s
```

4. `torch.cat()` / `torch.stack()`: Combining Tensors

```
a = torch.randn(3, 4)
b = torch.randn(3, 4)

# cat: Concatenate along an existing dimension
c = torch.cat([a, b], dim=0)   # [6, 4] — Stacked vertically
d = torch.cat([a, b], dim=1)   # [3, 8] — Stacked horizontally

# stack: Create a NEW dimension
e = torch.stack([a, b], dim=0) # [2, 3, 4] — New "batch" dimension
```

9️⃣ Broadcasting: The Semantic Helper

Sometimes we want to add a Vector to a Matrix. Mathematically, this is undefined (shapes don't match). PyTorch makes it work via **Broadcasting**.

```
A = torch.ones(3, 4) # Matrix
B = torch.tensor([1, 2, 3, 4]) # Vector

# Result: PyTorch "stretches" B to match A's shape
C = A + B
```

> [!TIP] **The Rule:** Dimensions are compared from the **right** (trailing dimensions first). At each position, sizes must be **equal** OR one of them must be **1**. PyTorch will copy the data along the "1" dimension to match the other tensor.

Broadcasting Rules in Action

```
# ☑ Example 1: Vector + Matrix (bias addition — happens in every nn.Linear!)
A = torch.ones(3, 4)           # [3, 4]
bias = torch.tensor([1, 2, 3, 4])  # [4] → stretched to [3, 4]
result = A + bias              # [3, 4] — bias added to EVERY row

# ☑ Example 2: Column vector + Row vector
col = torch.tensor([[1], [2], [3]])  # [3, 1]
row = torch.tensor([10, 20, 30])     # [3] → [1, 3]
result = col + row                    # [3, 3] — outer sum!

# ❌ Example 3: Incompatible shapes
# A = torch.ones(3, 4)   # [3, 4]
# B = torch.ones(3, 5)   # [3, 5]
# A + B  → RuntimeError! (4 ≠ 5, and neither is 1)
```

> [!NOTE] **Where Broadcasting Happens in Real Models:** Every `nn.Linear` layer does `y = x @ W.T + b`. The bias `b` has shape `[out_features]`, but `x @ W.T` has shape `[batch, out_features]`. Broadcasting automatically adds the bias to every sample in the batch.

🔟 The Big One: Autograd

Automatic Differentiation is the reason PyTorch exists.

🕸 The "No More Calculus" Promise

In the old days, you had to calculate derivatives by hand (Backprop).

- **Manual:** Painful, error-prone (`d_loss/dw = ...`).
- **Autograd:** You define the **forward** pass; PyTorch handles the **backward** pass.

🕹 Manual vs. Autograd

This is why we use frameworks.

Without PyTorch (Class 2):

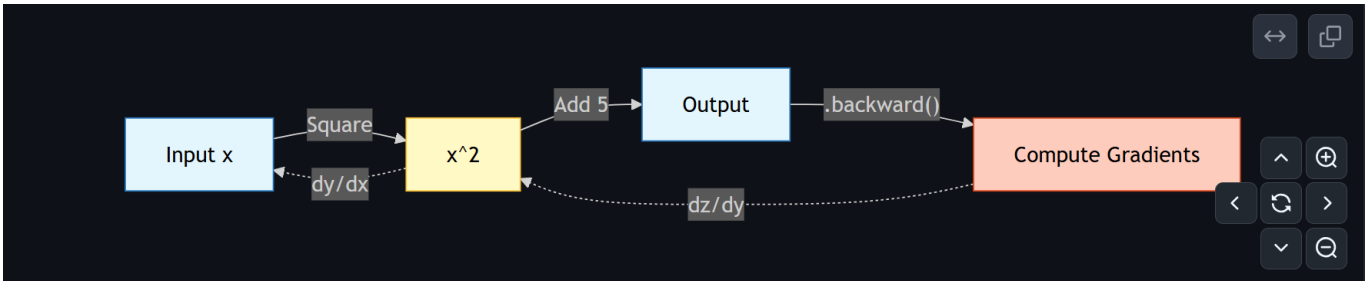
```
# Calculating gradients by hand... hard work! 😫
dL_dy = 2 * (y_pred - y_true)
dy_dw = x
dL_dw = dL_dy * dy_dw
w = w - lr * dL_dw
```

With PyTorch (Class 4):

```
# One line to rule them all. 🪄
loss.backward()
# PyTorch automatically computes dL/dw for EVERY parameter.
```

🕸️ The Computation Graph

PyTorch builds a graph dynamically as you run your code.




```
x = torch.tensor(2.0, requires_grad=True)
y = x**2 + 5
y.backward() # The Magic Line
print(x.grad) # Output: 4.0
```

> [!NOTE] **Why is this profound?** You can write *any* crazy python function with if-statements, loops, and recursion. As long as it uses PyTorch tensors, Autograd can differentiate through it. This is called **Define-by-Run**.

🔗 The Chain Rule in Pictures

Autograd uses the **chain rule** from calculus. If `y = f(g(x))`, then `dy/dx = dy/dg * dg/dx`.

```
# y = (x^2 + 1)^3
# dy/dx = 3*(x^2+1)^2 * 2x
# At x=1: dy/dx = 3*(2)^2 * 2 = 24
```

```
x = torch.tensor(1.0, requires_grad=True)
y = (x**2 + 1)**3
y.backward()
print(x.grad)  # 24.0 ✔ — Autograd handles the chain rule automatically!
```

🛡 `torch.no_grad()`: Save Memory During Inference

During inference (testing), you don't need gradients. Wrapping in `no_grad()` saves significant memory.

```
# During training: gradients ARE tracked
output = model(input_data)
loss = loss_fn(output, target)
loss.backward()  # ☑ Gradients flow

# During inference: gradients are NOT tracked
with torch.no_grad():
    predictions = model(test_data)  # Faster, uses less memory
    # loss.backward() would FAIL here
```

1️⃣1️⃣ Building Neural Networks: `nn.Module`

`nn.Module` is the LEGO baseplate for all Deep Learning models.

🏗 The Structure

Every model follows this exact pattern:

1. `__init__`: Define the layers (the parts).
2. `forward`: Define the flow (how parts connect).

```
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        # 1. The Parts
        self.layer1 = nn.Linear(784, 128) # Matrix: [128, 784]
        self.relu = nn.ReLU()             # Activation

    def forward(self, x):
        # 2. The Flow
        x = self.layer1(x) # x @ W.T + b
        x = self.relu(x)
        return x
```

🔍 What is `nn.Linear`?

It is NOT a black box. It is just **Matrix Multiplication + Bias**. y = x @ W.T + b

Why ReLU? (Why We Need Non-Linearity)

Without an activation function like ReLU, stacking multiple `nn.Linear` layers is pointless — they mathematically collapse into a **single linear transformation**.

```
# Without ReLU: Two layers = One layer (useless!)
# Layer 1: y = x @ W1 + b1
# Layer 2: z = y @ W2 + b2
# Combined: z = x @ (W1 @ W2) + (b1 @ W2 + b2)
#           z = x @ W_combined + b_combined  ← just ONE layer!

# With ReLU: The non-linearity breaks the collapsing!
# Layer 1: y = ReLU(x @ W1 + b1)   ← can't simplify past ReLU
# Layer 2: z = ReLU(y @ W2 + b2)   ← each layer adds REAL depth
```

ReLU (`max(0, x)`) is beautifully simple: keep positive values, kill negatives. This tiny non-linearity is what makes "deep" learning actually *deep*.

Inspecting What Your Model Learned

```
model = SimpleNet()

# See all learnable parameters
for name, param in model.named_parameters():
    print(f"{name}: {param.shape}")
    # layer1.weight: torch.Size([128, 784])
    # layer1.bias:   torch.Size([128])

# Count total parameters
total = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total:,}")  # 100,480
```

Common Loss Functions

Loss Function	Use Case	When to Use
`nn.MSELoss()`	Regression	Predicting continuous values (price, temperature).
`nn.CrossEntropyLoss()`	Multi-class Classification	Choosing 1 of N categories (digit 0-9).
`nn.BCEWithLogitsLoss()`	Binary Classification	Yes/No decisions (spam or not spam).

Common Optimizers

Optimizer	Description	When to Use

Optimizer	Description	When to Use
`SGD`	Vanilla gradient descent. Simple, predictable.	Baseline, academic experiments.
`Adam`	Adaptive learning rate per parameter. **The default choice.**	Almost everything. Start here.
`AdamW`	Adam with proper weight decay (regularization).	Production models, fine-tuning.

1️⃣2️⃣ The Pattern: How Models Learn

From MNIST to GPT-4, the "Training Loop" is identical. It is a 5-step heartbeat.

```
# 1. Forward Pass: The model predicts.
predictions = model(batch_x)

# 2. Compute Loss: How wrong was it?
loss = loss_fn(predictions, batch_y)

# 3. Backward Pass: Calculate gradients (Auto).
loss.backward()

# 4. Optimizer Step: Update weights (SGD/Adam).
optimizer.step()

# 5. Zero Gradients: Reset for next batch.
optimizer.zero_grad()
```

Step	Code	Meaning
1. Forward	`pred = model(x)`	"Guess the answer."
2. Loss	`loss = loss_fn(pred, y)`	"How wrong was I?"
3. Zero Grad	`optimizer.zero_grad()`	"Flush the memory (toilet)."
4. Backward	`loss.backward()`	"Calculate blame (gradients)."
5. Step	`optimizer.step()`	"Update weights (learn)."

> [!CAUTION] **The Classic Bug:** Forgetting `optimizer.zero_grad()`. If you omit this, PyTorch **accumulates** gradients. Your model won't learn; it will explode.

```
for epoch in range(10):
    y_pred = model(x_batch)      # 1. Forward
    loss = criterion(y_pred, y)  # 2. Loss

    optimizer.zero_grad()        # 3. Clean
```

```
    loss.backward()             # 4. Calculus
    optimizer.step()            # 5. Update
```

The Debugging Checklist

When your model isn't learning, check these first:

Symptom	Likely Cause	Fix
Loss is NaN	Learning rate too high, or exploding gradients.	Reduce LR, add gradient clipping.
Loss not moving	LR too low, or `zero_grad()` missing.	Increase LR, add `zero_grad()`.
Shape Mismatch	Tensor dimensions don't align.	Print `.shape` before every layer!
Silent Failure	Used Python `list` instead of `Tensor`.	Always use `torch.tensor()`.
Model on wrong device	Data on CPU, model on GPU (or vice-versa).	Check `.device` on both.
Loss stuck at random	Forgot to call `model.train()`.	Always `model.train()` before training.

Key Terminology

Term	Definition
Epoch	One complete pass through the entire dataset.
Batch	A subset of the data processed at once (e.g., 32 samples).
Iteration	One forward + backward pass on a single batch.
Learning Rate	How big of a step to take when updating weights. Too high = overshoot, too low = stuck.

1️⃣3️⃣ Final Recap: The "Deep" in Deep Learning

We started with scalars and ended with a Neural Network. Let's trace the journey of a single image through the **Transformation Pipeline**.

1. The Raw Data (784 Dimensions)

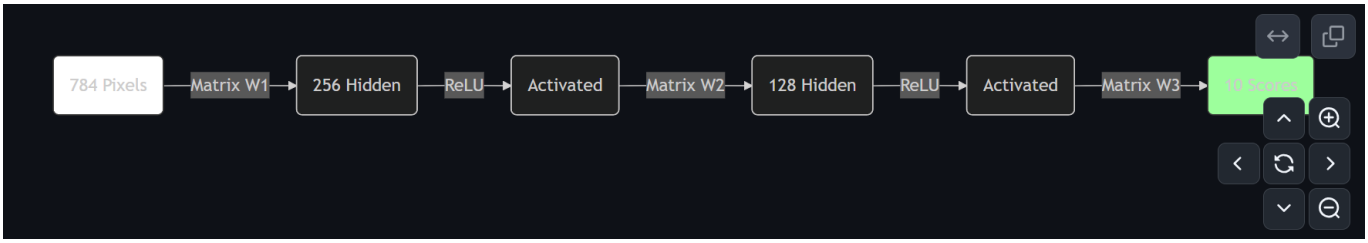
A 28x28 image is flattened into a vector of **784** numbers. This is our input space.

2. The Transformations (The "Layers")

We don't go straight to the answer (0-9). We transform the data through intermediate dimensions ("Hidden Layers").

- **784 -> 256:** We project the raw pixels into a 256-dimensional space.

- **256 -> 128:** We summarize those features into 128 conceptually higher-level features.
- **128 -> 10:** Finally, we crush it down to 10 probabilities (one for each digit).



3. The Code Implementation

This entire pipeline is defined in fewer than 15 lines of PyTorch code.

```
class DigitClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()             # 28x28 -> 784
        self.layer1 = nn.Linear(784, 256)       # Transform 1
        self.layer2 = nn.Linear(256, 128)       # Transform 2
        self.layer3 = nn.Linear(128, 10)        # Transform 3 (Output)
        self.relu = nn.ReLU()                   # Non-linearity

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu(self.layer1(x))
        x = self.relu(self.layer2(x))
        x = self.layer3(x)
        return x
```

The Big Picture

- **Tensors** hold the data.
- **Matrices** transform the data (rotate, scale, shear).
- **Autograd** tweaks the matrices so the transformations become useful.
- **GPUs** make it happen effectively instantly.

This is the Deep Learning Engine.

PyTorch Quick Reference Card

The 15 most important things you'll use daily.

Category	Code	What It Does
Create	`torch.randn(3, 4)`	Random tensor, shape [3,4]
Create	`torch.zeros(3, 4)`	All zeros
Shape	`x.shape`	Get dimensions

Category	Code	What It Does
Reshape	`x.view(12)`	Flatten (zero-copy)
Reshape	`x.unsqueeze(0)`	Add batch dimension
Combine	`torch.cat([a, b], dim=0)`	Join tensors
MatMul	`A @ B`	Matrix multiplication
Device	`x.to("cuda")`	Move to GPU
Grad	`x.requires_grad_(True)`	Enable gradient tracking
Grad	`loss.backward()`	Compute all gradients
Grad	`optimizer.zero_grad()`	Reset gradient buffer
Grad	`optimizer.step()`	Update weights
Inference	`torch.no_grad()`	Disable grad tracking
Debug	`print(x.shape, x.dtype, x.device)`	The holy trinity of debugging
Model	`model.parameters()`	Access all learnable weights

Made with ♥ by Akshat Shethia